Project Specification: Custom Sports Analytics Database & API

Prepared for: Independent Software Engineer / Contractor

Data Source Backend: Punting Form Public API V2 (Professional Tier)

Primary Filter Focus: Western Australian (WA) Racing (with initial emphasis on Saturday Ascot Meetings)

> **Architectural Core Directive: "Store Everything, Filter Later"**
> The database must be built as a raw data warehouse. The ingestion pipeline must capture and store complete, unedited row payloads from the Punting Form API endpoints. Changing or adding filtering logic down the track must only require modifying the view/API layer, completely preserving the underlying data architecture.

1. Scope of Work & Target Deliverables

- **Daily ETL Ingestion Pipeline:** An automated Python script running via cron job that authenticates against Punting Form V2 API, checks for target meetings (e.g., Saturday Ascot), and processes raw fields into a local warehouse.
- **Relational Database:** A optimized PostgreSQL instance designed to map structured racing entities and rapidly index sequential fractional data.
- **Lightweight API Layer:** A secure REST API built using FastAPI or Flask to expose modular, parameterised endpoints for automated form filtering.
- **Cross-Device User Interface:** Responsive HTML table outputs or automated text webhooks (e.g., Telegram Bot) to allow secure remote access via any smartphone or computer.

2. Database Entity & Schema Requirements

The database schema must separate data into clean relational tables to handle sequential parameters efficiently:

- `meetings` : Tracks static daily parameters: Date, Track Name, Rail Position, Track Condition, and Moisture Variant.
- `races` : Tracks Race Number, Distance, Class Grade, and Class Par values.
- `runners` : Tracks Horse Name, Barrier, Weight, Trainer, Jockey, Predicted Settle (`PrSe`), Run Style (`RunStyle`), and Key Race Index (`KRI`).

- `sectionals_200m` : A sequential table mapping every individual 200m chunk run by every horse. Must record Sectional Increments, Sectional Time, Raw Last 600m/200m, and the Tempo-Adjusted Benchmark Variables.

3. Crucial Analytical Variable Mappings

Web Interface Term	API Backend Variable	Database Function & Purpose
Key Race Index	`KRI`	Calculates subsequent form lines on a scale of 0-100. Measures race strength.
Class Change / Diff	`ClsDiff`	Tracks numeric class drops (negative value) or rises (positive value).
Var L 600 (L600)	`FML600`	Fastest Meantime Lengths for last 600m. Tempo-adjusted efficiency rating.
Var L 200 (L200)	`FML200`	Fastest Meantime Lengths for final 200m. Identifies peak terminal sprint velocity.
Runs Back	`RB`	Preparation counter. `0` signifies first-up from a spell, `1` signifies second-up.
Days Since Last Run	`DLR`	Tracks the exact space between starts to evaluate fitness cycles and quick backups.

4. Custom Algorithmic Filtering Rules

The developer must write modular endpoints that allow dynamic parameter passing. The core filters required for day-one deployment include:

A. The Automated Class Dropper & Hot Form Scanner

Isolates runners stepping down significantly in competitive grade who competed in high-pressure historical fields.

```
# Target Logic Sample
system_bets = df[(df['ClsDiff'] < 0) & (df['KRI'] >= 70)]
```

B. The Elite Closing Sectional Matrix

Isolates backmarkers or mid-fielders who clocked elite finishing terminal velocity lengths (FML) regardless of whether they won or missed a place visually.

```
# Target Logic Sample
elite_closers = df[(df['FML200'] >= 2.5) & (df['RunStyle'].isin(['Mid-Field',
'Backmarker']))]
```

C. Custom Dynamic Trainer Strike Rates

The system must query historical warehouse entries to aggregate custom trainer trends dynamically based on current parameters:

- **1st-Up Pattern:** Group by `Trainer` where `RB == 0` to evaluate fresh-up success percentages.
- **The 7-Day Backup:** Group by `Trainer` where `DLR <= 7` to expose trainers with high ROI when running horses on tight backup schedules.

5. Project Delivery & Payment Milestones

To protect project budget and ensure compliance with Punting Form API structures, payments should be broken down across the following four clear milestones:

Milestone	Delivery Target	Verification Requirement
Milestone 1	API Connection & PostgreSQL Ingestion Pipeline Script completed.	Successful data dump of target raw JSON/CSV feeds into database local tables.
Milestone 2	Database Architecture & Sectional/Variable Table Schema optimization.	Successful indexing of sequential 200m tables with filter targets limited to WA tracks.
Milestone 3	API Layer Development (FastAPI/Flask) with custom parameter filtering.	Successful live query test returning correct Class Dropper and FML data arrays.
Milestone 4	Cloud Server Deployment, Secure Authentication setup, and ReadMe handover.	System accessible securely via mobile browser/link with functional HTML data layout.